

동시성을 위한 Go

1편: 동시성 소개

남궁명수

[01-1] 무어의 법칙, 웹 스케일, 그리고 우리가 마주한 혼돈	3
(1) 암달의 법칙	3
(2) 수평 확장과 클라우드 컴퓨팅(Horizontal Scaling and Cloud Computing)	4
(3) 웹 스케일의 시대와 개발자의 숙제(Web Scale)	4
(4) 결론: 소프트웨어의 공짜 점심은 끝났다.	5
[01-2] 동시성이 왜 어려운가	6
(1) 데이터 경쟁(Race Conditions)	6
(2) 원자성(Atomicity)	7
(3) i++ 문장의 이면(쪼개지는 단편)	8
(4) 결론	8
[01-3] 메모리 접근 동기화	9
(1) 임계영역이란 무엇인가?	9
(2) 메모리 접근 동기화 구현하기	9
(3) 동기화의 한계와 부작용	10
(4) 결론	11
[01-4] 데드락, 라이브락, 기아 상태	12
(1) 교착 상태(Deadlock)	12
(2) 라이브락(Livelock)	13
(3) 기아 상태(Starvation)	14
(4) 균형 찾기	15
[01-5] 동시성 안전성 판단	16
(1) 동시성 설계라는 예술	16
(2) 모호한 코드와 소통의 부재	16
(3) 주석의 힘과 3가지 핵심 요소	16
(4) 코드 설계(시그니처)를 통한 신호 전달	17
(5) 결론	17
[01-6] 복잡함 속에서 단순함	18
(1) 절망 속에서 찾는 희망	18
(2) 메모리 관리의 해방: 저지연 가비지 컬렉터	18
(3) 스레드 관리의 자동화: 고루틴의 멀티플렉싱	19
(4) 거대한 문제를 쉽게 조립하는 채널	19
(5) 결론	20

[01-1] 무어의 법칙, 웹 스케일, 그리고 우리가 마주한 혼돈

1965년, 고든 무어는 전자공학 사장이 ‘집적 회로(Integrated Circuit, IC칩)’ 중심으로 통합될 것이며, 하나의 칩에 들어가는 부품의 수가 최소 10년 동안 매년 두 배씩 늘어날 것이라는 고작 3쪽짜리 논문을 발표했다. 이후 1975년에 그는 이 예측을 조금 수정해서 “마이크로칩에 저장할 수 있는 데이터의 양(부품 수)이 약 2년마다 두 배씩 증가할 것”이라고 선언했다. 이 예측이 그 유명한 ‘무어의 법칙’이며, 비교적 최근인 2012년 무렵까지 거의 정확하게 맞아떨어졌다.

하지만 여러 기술 기업들은 이 무어의 법칙이 언젠가 물리적인 한계에 부딪혀 성장 속도가 느려질 것을 진작에 예상했다. 그리고 컴퓨터의 처리 능력을 올릴 다른 대안들을 찾기 시작했다. “필요는 발명의 어머니”라는 말처럼, 기술적인 한계를 극복하기 위해 마침내 하나의 CPU 안에 여러 개의 두뇌를 넣는 ‘멀티코어 프로세서’가 탄생하게 되었다.

이 멀티코어의 등장은 무어의 법칙이 가로막은 벽을 넘어설 아주 똑똑한 해결책처럼 보였다. 하지만 컴퓨터 과학자들은 곧바로 또 다른 법칙의 한계에 부딪히게 되었다. 컴퓨터 구조학자 진 암달의 이름을 딴 ‘암달의 법칙’이 바로 그것이다.

(1) 암달의 법칙

암달의 법칙은 어떤 문제를 여러 개로 쪼개어 동시에 처리(병렬화)할 때, 이론적으로 최대 얼마나 빨라질 수 있는지 모델링하는 공식이다. 쉽게 말해 이 법칙은 다음과 같은 진리를 말해준다.

암달의 법칙 핵심

“아무리 CPU 코어 수를 늘려도, 프로그램 내에서 반드시 순서대로만 처리해야 하는 부분(순차적 영역)이 있다면 전체 속도 향상은 그 순차적 영역의 비율에 의해 제한된다.”

예를 들어, 사용자가 눈으로 화면을 보고 버튼을 클릭하면 무언가 작동하는 GUI(그래픽 사용자 인터페이스) 기반 프로그램을 만든다고 상상해보자. 이 프로그램은 파이프라인 중 가장 거대하고 도저히 줄일 수 없는 순차적인 단계에 가로 막혀있다.

바로 ‘사람의 상호작용(인간이 고민하고 마우스를 움직이는 시간)’이다. 이런 프로그램은 컴퓨터에 코어를 100개, 1000개 달아주더라도 사용자가 버튼을 눌러주는 속도 이상으로 절대로 빨라질 수 없다.

반대로 다른 예시를 생각해보겠다. 원주율의 자릿수를 구하는 계산이다. ‘스피곳 알고리즘’같은 특수한 방식을 사용하면, 이 문제는 각각의 계산을 서로 완전히 독립적으로 쪼개서 돌릴 수 있다. 컴퓨터 과학에서는 이를 ‘대체 없이 병렬적인 문제’라는 아주 독특한 전문 용어로 부른다. 이름은 좀 장난스럽지만 “너무 쉽게 병렬로 쪼갤 수 있어서 민망할 정도의 문제”라는 뜻이다. 이런 문제라면 코어를 늘리는 만큼 프로그램 속도가 엄청나게 빨라진다. 이제 개발자의 숙제는 ‘그 수많은 코어들이 계산해 온 결과물들을 어떻게 잘 합치고 저장할 것인가’로 바뀔 뿐이다.

암달의 법칙은 이처럼 우리가 해결하려는 문제의 성격을 이해하도록 도와주며, 우리 시스템의 성능을 올리기 위해 ‘병렬화(동시성)’를 도입하는 게 과연 올바른 선택인지 판단할 수 있는 기준이 되어준다.

(2) 수평 확장과 클라우드 컴퓨팅(Horizontal Scaling and Cloud Computing)

‘대책 없이 병렬적인’ 문제를 다룰 때는, 애플리케이션을 ‘수평 확장(scale out)’할 수 있도록 설계하는 것이 정석이다.

수평 확장이란 똑같은 프로그램 인스턴스를 여러 개 복사해서 더 많은 CPU나 더 많은 컴퓨터에서 동시에 돌리는 것을 말한다. 문제가 쉽게 쪼개지기 때문에, 여러 컴퓨터에 일을 한 덩어리씩 던져 주기만 하면 되므로 시스템 전체의 실행 시간을 극적으로 줄일 수 있다.

이러한 수평적 확장은 2000년대 초반, 전 세계 IT 업계를 뒤흔든 새로운 패러다임 덕분에 훨씬 쉬워졌다. 바로 ‘클라우드 컴퓨팅’이다. (개념 자체는 1970년대에도 있었지만, 대중의 마음속에 완전히 뿌리 내린 건 2000년대 초반이다.)

클라우드 컴퓨팅은 애플리케이션을 배포하고 확장하는 방식으로 완전히 바꾸어 놓았다. 과거에는 개발자가 컴퓨터 부품을 직접 고르고, 운영체제와 소프트웨어를 조심스레 설치하며 애지중지 관리해야 한다. 하지만 클라우드 세상에서는 거대한 자원 풀(Pool)에 접근해, 필요할 때마다 원하는 성능의 가상 컴퓨터를 ‘주문형(On-demand)’으로 즉시 생성해서 쓸 수 있게 되었다. 컴퓨터는 이제 한 번 사면 평생 쓰는 물건이 아니라, 잠시 쓰고 버릴 수도 있는 ‘일시적인 존재’가 되었고, 내 프로그램의 특성에 딱 맞춘 스펙으로 언제든지 커스텀할 수 있게 되었다. 일반적으로 이 거대한 컴퓨터 자원들은 거대 기술 기업들의 데이터 센터에 보관되어 있다.

이 변화는 개발자들의 사고방식을 완전히 바꾸어 놓았다. 갑자기 개발자들은 대형 슈퍼컴퓨터 급의 엄청난 컴퓨팅 파워를 아주 저렴한 비용으로 손에 쥐게 되었다. 이제 하나의 서비스가 수많은 컴퓨터를 넘어 전 세계 여러 지역(Region)으로 손쉽게 뻗어 나갈 수 있게 될 것이다. 예전에는 구글이나 아마존 같은 테크 대기업들만 풀 수 있었던 거대한 규모의 문제들을, 이제는 누구나 클라우드를 통해 해결할 수 있게 되었다.

(3) 웹 스케일의 시대와 개발자의 숙제(Web Scale)

물론 클라우드 컴퓨팅이 축복만 가져다준 것은 아니다. 수많은 컴퓨터 자원을 자동으로 준비(Provisioning)하고, 수십 수백 대의 컴퓨터들끼리 서로 네트워크 통신을 주고받게 만들며, 각자 계산한 결과를 다시 하나로 예쁘게 모아서 저장하는 것 모두가 해결해야 할 거대한 숙제가 되었다.

하지만 그 중에서도 가장 어려웠던 것은 ‘코드를 동시성 있게(주변 컴퓨터들과 조화롭게 동시에 돌아가도록) 모델링 하는 방법’을 찾아내는 일이었다.

내 프로그램의 파편들이 전 세계 여기저기에 흩어진 서로 다른 컴퓨터에서 돌아가다 보니, 기존 동시성 프로그래밍이 가졌던 고질적인 문제들이 훨씬 더 심각하고 복잡하게 터지기 시작한 것이다.

이 난제들을 성공적으로 해결해 낸 소프트웨어들에게는 아주 멋진 훈장이 붙기 시작했는데, 그것이 바로 ‘웹 스케일’이라는 브랜드이다.

웹 스케일 소프트웨어란?

수십만 명(혹은 그 이상)의 사용자가 동시에 몰려와 엄청난 작업량이 발생하더라도, 시스템을 끄지 않고 단순히 애플리케이션 인스턴스(컴퓨터)를 옆으로 계속 추가하는 것만으로도 무한히 버텨낼 수 있는 소프트웨어를 뜻한다.

소프트웨어가 웹 스케일 수준에 도달하면, 서비스를 멈추지 않고 업데이트하는 ‘무중단 배포’, 트래픽에 따라 컴퓨터가 자동으로 늘어나고 줄어드는 ‘탄력적 수평 확장 구조’, 전 세계 인프라 분산 등이 가능해진다. 하지만 그 대가로 시스템을 머릿속으로 이해하기가 수만 배 더 어려워졌고, 컴퓨터 한두 대가 고장 나도 멈추지 않게 만드는 ‘결함 허용’의 난이도도 극악으로 높아졌다.

(4) 결론: 소프트웨어의 공짜 점심은 끝났다.

결국 현대의 개발자들은 멀티코어 CPU, 클라우드 컴퓨팅, 웹 스케일, 그리고 병렬화가 될지 안될지 모르는 복잡한 문제들이 뒤엉킨 거대한 혼돈의 한복판에 서게 되었다. 조금은 압도당할 수밖에 없는 환경이다.

과거에는 가만히 앉아만 있어도 하드웨어(CPU)가 알아서 빨라져서 내 프로그램도 같이 빨라졌지만, 이제 그 ‘좋았던 시절’은 끝났다. 바통은 온전히 우리 개발자들에게 넘어왔고, 우리는 우리 손에 쥐어진 멀티코어라는 하드웨어 한계 속에서 동시성 코드를 짜며 문제를 돌파해야만 한다.

2005년, 허브 서터는 <Dr.Dobb's> 저널에 “공짜 점심은 끝났다: 소프트웨어 동시성을 향한 근본적인 전환”이라는 역사적인 글을 기고했다. 이 제목은 아주 적절했고, 그의 미래 예측은 정확했다. 그는 글의 마지막 부분에서 이렇게 강조했다.

허브서터의 예언

“우리에게는 지금의 프로그래밍 언어들이 제공하는 수준보다, 훨씬 더 높은 차원에서 동시성을 제어할 수 있는 프로그래밍 모델이 절실하게 필요하다.”

그가 왜 이렇게 강력한 어조로 경고했는지, 도대체 동시성 프로그래밍을 똑바로 짜는 것이 왜 그렇게 어려운 일인지, 이제 알아보자.

[01-2] 동시성이 왜 어려운가

(1) 데이터 경쟁(Race Conditions)

경쟁 조건이란 두 개 이상의 작업이 ‘정확한 순서’대로 실행되어야만 프로그램이 올바르게 작동하는데, 프로그램 자체에는 그 순서를 보장하는 장치가 없어서 발생하는 문제이다.

대부분의 경우 이는 ‘데이터 경쟁(Data Race)’이라는 형태로 나타난다. 즉, 하나의 동시성 작업(고루틴)이 어떤 변수를 읽으려고(Read)하는 바로 그 타이밍에, 다른 고루틴이 그 변수에 무언가를 쓰려고(Write) 격렬하게 경쟁하는 상황이다.

데이터 경쟁의 기초 예시

```
var data int

go func() {
    data++ // 고루틴이 값을 1 증가시킴 (Line 3)
}()

if data == 0 { // 메인 루프가 값을 확인 (Line 5)
    fmt.Printf("the value is %v.\n", data) // (Line 6)
}
```

“Go 언어에서는 함수 앞에 go 키워드를 붙이면 그 함수를 동시에 실행하는 독립적인 일꾼인 ‘고루틴’이 생성된다.”

위 코드를 실행하면 아주 무서운 일이 벌어진다. 3번 라인과 5번 라인이 동시에 data라는 변수에 접근하는데, 누가 먼저 도착할지 아무도 보장할 수 없다. 이 코드를 실행했을 때 나오는 결과는 다음 3가지 중 하나가 된다.

- 아무것도 출력되지 않음: 고루틴이 엄청나게 빨리 일해서 메인 함수가 if data == 0을 검사하기 전에 이미 값을 1로 올려버리는 경우
- “the value is 0”이 출력됨: 메인 함수가 너무 빨라서 고루틴이 일을 시작하기도 전에 검사(5번)와 출력(6번)을 모두 끝내 버리는 경우이다.
- “the value is 1”이 출력됨: 메인 함수가 if data == 0 검사를 할 때는 분명 0이었는데, 바로 다음 줄인 fmt.Printf 출력을 실행하기 직전에 고루틴이 끼어들어 값을 1로 올려버린 경우이다.

사람이 가진 순차적 사고의 오류

개발자들이 데이터 경쟁 버그를 자주 만드는 이유는 머릿속으로 코드를 ‘순차적’으로만 생각하기 때문이다. 소스코드 상에서 윗줄에 있으니 당연히 먼저 실행되겠지 착각하는 것이다.

동시성 코드를 짤 때는 가능한 모든 시나리오를 꼼꼼하게 따져봐야 한다.

컴퓨터의 시간 감각이 와닿지 않을 때는 “각 줄의 코드 사이에 1시간씩 공백이 있다”고 극단적으로 상상해보는 것이 도움이 된다. 고루틴을 켜고 나서 1시간 뒤에 고루틴이 작동한다면? 혹은 if문을 검사하고 1시간 뒤에 출력문으로 내려간다면? 컴퓨터의 세계에서는 실제로 이 정도의 상대적인 시간 차이가 빈번하게 일어난다.

‘Sleep’이라는 달콤한 함정

어떤 개발자들은 이 문제를 해결하겠다고 코드 중간중간에 슬립(time.Sleep)을 뿌려두는 함정에 빠진다.

```
var data int
go func() { data++ }()

time.Sleep(1*time.Second) // 매우 나쁜 코드!

if data == 0 {
    fmt.Printf("the value is %v.\n", data)
}
```

슬립을 1초 주었으니 고루틴이 먼저 덧셈을 끝낼 확률이 훨씬 높아진 것은 사실이다. 하지만 이 코드는 여전히 근본적으로 틀렸다. 컴퓨터가 순간적으로 느려지면 1초 뒤에도 경쟁이 발생할 수 있다. 올바른 도구를 쓰지 않고 슬립에 의존하는 것은 올바른 정답(논리적 해결성)에 근접할 뿐, 절대 완벽한 정답이 될 수 없다. 게다가 무의미하게 1초를 기다려야 하므로 성능도 엄청나게 떨어진다.

경쟁 조건이 정말 무서운 이유는 서비스 오픈 후 몇 년 동안 조용하다가, 하드웨어가 바뀌거나 트래픽이 몰리는 순간 갑자기 나타나 시스템을 폭파시키기 때문이다.

(2) 원자성(Atomicity)

어떤 작업이 원자적이다. 혹은 원자성을 가진다는 것은 그 작업이 실행되는 문맥(Context) 안에서 더 이상 쪼갤 수 없고, 중간에 가로막거나 끊을 수 없다는 뜻이다.

원자성에서 가장 중요한 것은: 문맥(Context)

원자성을 논할 때는 반드시 그 작업이 일어나는 범위(Scope)나 문맥을 먼저 정의해야 한다. 어떤 작업이 ‘내 프로그램 안’에서는 원자적일지 몰라도 ‘운영체제(OS) 관점’에서는 아닐 수 있고, ‘OS 관점’에서는 원자적이어도 ‘하드웨어(CPU) 관점’에서는 쪼개질 수 있기 때문이다.

재미있는 일화: 월드 오브 워크래프트와 Glider 봇(2006)

블리자드 사의 게임 와우(WoW)에는 ‘와든(Warden)’이라는 해킹 방지 프로그램이 있었다. 와든은 컴퓨터의 메모리를 쭉 스캔하면서 해킹 프로그램을 찾아냈는데, 와든 프로세스 입장에서는 메모리를 스캔하는 작업이 완벽하게 끊김없는 하나의 원자적 작업이었다.

하지만 ‘Glider’라는 오토 봇 프로그램은 하드웨어 인터럽트(OS보다 더 하위 개념)를 가로채서, 와든이 스캔을 시작하기 직전에 메모리에서 자신을 쭉 숨겨버렸다. 즉, 와든의 스캔은 자신의 프로그램 안(Process Context)에서는 원자적이었지만, 운영체제와 하드웨어 관점에서는 쪼갤 수 있는 작업이었던 것이다. 이 틈을 찌른 것이다.

(3) i++ 문장의 이면(쪼개지는 단편)

우리 코드에서 가장 단순해 보이는 i++(값을 1증가시키기) 문장을 예로 들어 원자성을 살펴볼 것이다. 눈으로 보기엔 한 줄짜리 원자적 작업 같지만, 컴퓨터 내부에서는 사실 3가지 단계로 쪼개져서 실행된다.

1. 현재 i의 값을 메모리에서 가져온다
2. 가져온 값을 1증가시킨다.
3. 증가된 새 값을 다시 메모리에 저장한다.

각각의 1, 2, 3번 행동 자체는 원자적이지만, 이 3개를 합쳐 놓은 i++ 전체는 동시성 환경에서 원자적이지 않다. 만약 1번과 3번 사이에 다른 고루틴이 끼어들어서 i값을 바꿔버리면 데이터가 오염된다.

반대로, 만약 이 코드가 동시성(고루틴)이 전혀 없는 순차적인 프로그램이거나, 다른 고루틴과 절대로 공유하지 않는 독립된 고루틴 내부의 변수라면 이 i++은 그 문맥 안에서 안전한 ‘원자적 작업’이 된다.

(4) 결론

우리가 원자성에 집착하는 이유는 명확하다. 어떤 작업이 완벽하게 원자적이라면, 여러 고루틴이 동시에 접근해도 완전히 안전하기 때문이다.

하지만 안타깝게도 우리가 쓰는 대부분의 코드와 함수는 원자적이지 않다. 따라서 동시성 프로그래밍을 잘한다는 것은 내 코드에서 어떤 부분을 원자적으로 강제 결합해야 하는지, 그리고 얼마나 작은 단위로 그 범위를 쪼개고 묶어야 하는지를 찾아내는 예술과도 같다. 이 결합을 도와주는 다양한 도구들과 테크닉들을 앞으로 배워보자.

[01-3] 메모리 접근 동기화

(1) 임계영역이란 무엇인가?

두 개 이상의 동시성 프로세스(고루틴)가 동일한 메모리 영역에 접근하는데, 그 방식이 원자적이지 않을 때 데이터 경쟁이 발생한다고 배웠다. 이전 예제를 살짝 수정해서, 어떤 상황에서도 무조건 화면에 출력되도록 else 문을 추가해보겠다.

```
var data int

go func() {
    data++ // 고루틴이 값을 1 증가시킴
}()

if data == 0 {
    fmt.Println("the value is 0.")
} else {
    fmt.Printf("the value is %v.\n", data)
}
```

이 코드는 여전히 데이터 경쟁을 가지고 있으며, 실행할 때마다 결과가 달라지는 비결정적인 상태이다.

여기서 아주 중요한 컴퓨터 과학 용어가 등장한다. 프로그램 내부에서 공유 자원에 독점적으로 접근해야 하는 안전 구역을 ‘임계 영역’ 이라고 부른다. 위 코드에는 총 3개의 임계 영역이 존재한다.

- data++: 공유변수 data를 읽고 수정하는 고루틴 영역
- if data == 0: data의 값이 0인지 확인하기 위해 메모리를 읽는 영역
- fmt.Printf(...): 출력하기 위해 data의 값을 메모리에서 가져오는 영역

이 임계 영역들을 안전하게 보호하는 방법 중 가장 대표적인 것이 바로 ‘메모리 접근 동기화’이다.

(2) 메모리 접근 동기화 구현하기

아래 코드는 사실 Go 언어에서 권장하는 스타일은 아니다. (실제 데이터 경쟁을 풀 때 이렇게 짜는 것은 추천하지 않는다.) 하지만 ‘메모리 접근 동기화’가 어떤 원리로 작동하는지 가장 직관적으로 보여주는 예시이다.

문법이 조금 낯설더라도 “메모리 접근 순서를 조율하는 개념”에 집중해서 행간의 설명을 따라가자.

```
var memoryAccess sync.Mutex // 1. 메모리 접근을 통제할 '자물쇠' 변수 생성
var value int

go func() {
    memoryAccess.Lock() // 2. 고루틴이 자물쇠를 잠금 (독점 권한 획득)
    value++
    memoryAccess.Unlock() // 3. 불일이 끝났으니 자물쇠를 열어줌
}()

memoryAccess.Lock() // 4. 메인 함수가 자물쇠를 잠금 (독점 권한 획득)
if value == 0 {
    fmt.Printf("the value is %v.\n", value)
}
```

```

} else {
    fmt.Printf("the value is %v.\n", value)
}
memoryAccess.Unlock() // 5. 메인 함수가 볼일을 마쳤으니 자물쇠를 열어줌

```

코드의 작동 원리

1. sync.Mutex

공유 변수의 메모리에 아무나 접근하지 못하도록 순서를 맞춰주는 ‘자물쇠(Mutex)’ 역할을 한다.

2. memoryAccess.Lock() - 고루틴 내

“지금부터 내가 이 메모리를 독점하겠어!”라고 선언하는 것이다. 만약 다른 곳에서 이미 잠갔다면, 풀릴 때까지 이 자리에서 서서 기다린다.

3. memoryAccess.Unlock() - 고루틴 내

“나 이제 메모리 다 썼으니 다른 사람 쓰세요”라고 권한을 반납하는 것이다.

4. memoryAccess.Lock() - 메인 함수 내

아래 조건문(if-else)들이 안전하게 value 변수를 읽을 수 있도록 메인 함수 측에서 독점 권한을 요청한다.

5. memoryAccess.Unlock() - 메인 함수 내

출력이 끝났으니 자물쇠를 열어준다.

이처럼 개발자들 사이에 규칙(컨벤션)을 정한 것이다.

“변수를 건드리고 싶으면 무조건 Lock()을 먼저 부르고, 끝나면 Unlock()을 부르자”

이 규칙을 지키면 두 문장 사이의 코드는 완벽한 독점권을 보장받으며, 메모리는 안전하게 동기화된다.

주의할 점: 만약 어떤 개발자가 깜빡하고 이 규칙을 따르지 않은 채 value 변수를 그냥 가져다 쓰면, 이 모든 독점권 보장 체계는 순식간에 물거품이 된다.

(3) 동기화의 한계와 부작용

“자물쇠만 잘 채우면 동시성 버그는 다 끝나는 거 아닌가?”라고 생각할 수 있지만, 현실은 그리 만만하지 않다. 메모리 접근 동기화에는 몇 가지 치명적인 한계와 부작용이 있다.

- 데이터 경쟁은 풀었지만 경쟁 조건은 풀지 못했다.

자물쇠를 채워서 메모리가 오염되는 ‘데이터 경쟁’은 막았지만, 누가 먼저 실행될지 모르는 ‘경쟁 조건’은 전혀 해결되지 않았다. 고루틴이 먼저 자물쇠를 쥐고 값을 1로 올릴지, 아니면 메인 함수가 먼저 자물쇠를 쥐고 if문을 실행할지 여전히 알 수 없다.

단지 비결정적인 범위가 지금 좁혀졌을 뿐이다.(이 진짜 문제를 제대로 푸는 도구는 뒤에서 배운다.)

- 순진한 규칙이 가진 위험성

이 방식은 “모든 개발자가 항상 자물쇠를 잘 잠그고 열 것”이라는 신뢰를 바탕으로 작동한다. 하지만 바쁜 비즈니스 요구사항에 쫓기다 보면 누군가는 규칙을 누락하기 마련이다. 사람이 조심하는 것에는 한계가 있다.

- 성능 저하의 주범

자물쇠를 잠그는 Lock() 행동은 비용이 꽤 크다. 프로그램이 Lock()을 만날 때마다 다른 고루틴들은 하던 일을 멈추고 제자리에 서서 기다려야하기 때문이다. 이 때문에 프로그램이 급격히 느려질 수 있다.

따라서 개발자는 늘 다음 두 가지 질문 사이에서 줄타기를 해야 하는 예술적인 고민에 빠지게 된다.

- 내 프로그램이 이 임계 영역을 너무 자주 들락날락 거리며 성능을 갉아먹고 있지 않은가?
- 임계 영역의 크기(자물쇠를 잠그고 있는 코드의 길이)를 얼마나 크게 혹은 작게 잡아야 안전하면서도 빠를까?

(4) 결론

임계 영역이 보인다고 해서 무조건 자물쇠(Mutex)부터 채우는 것이 정답은 아니다. 메모리 동기화는 동시성 문제를 해결하는 하나의 방법일 뿐, 시스템의 정답과 성능을 보장해 주지는 않는다.

게다가 이 자물쇠 방식은 동시성 프로그래밍의 또 다른 거대한 재앙인 ‘데드락’이나 ‘라이브락’ 같은 고질적인 문제들을 유발하기도 한다.

[01-4] 데드락, 라이브락, 기아 상태

(1) 교착 상태(Deadlock)

데드락(교착 상태)이란 모든 동시성 프로세스(고루틴)가 서로 상대방이 가진 자원을 기다리며 영원히 멈춰버리는 상태를 말한다. 이 상태에 빠지면 외부에서 프로그램을 강제로 종료하고 다시 켜지 않는 이상 절대 스스로 회복될 수 없다.

Go언어의 런타임은 프로그램 내의 모든 고루틴들이 잠들어 버리는 전면적 데드락을 감지하면 프로그램을 강제로 종료(fatal error)시켜 주지만, 아쉽게도 일부 고루틴만 멈추는 부분 데드락은 잡아내지 못하므로 우리가 설계할 때부터 조심해야 한다.

데드락의 대표적인 예시

```
type value struct {
    mu sync.Mutex
    value int
}

var wg sync.WaitGroup

printSum := func(v1, v2 *value) {
    defer wg.Done()

    v1.mu.Lock()      // 1. 첫 번째 자물쇠를 잠금
    defer v1.mu.Unlock()

    time.Sleep(2*time.Second) // 2. 데드락을 확실히 유발하기 위한 가상 딜레이

    v2.mu.Lock()      // 3. 두 번째 자물쇠를 잠금 (여기서 대기!)
    defer v2.mu.Unlock()

    fmt.Printf("sum=%vWn", v1.value + v2.value)
}

var a, b value
wg.Add(2)
go printSum(&a, &b) // 고루틴 1: a를 잠그고 b를 얻으려 함
go printSum(&b, &a) // 고루틴 2: b를 잠그고 a를 얻으려 함
wg.Wait()
```

이 코드를 실행하면 즉시 다음과 같은 에러를 내며 뺏어버린다.

```
fatal error: all goroutines are asleep - deadlock!
```

- 원인 분석

두 고루틴의 타이밍이 꼬였다. 첫 번째 고루틴이 a를 잠그는 동안, 두 번째 고루틴은 b를 잠갔다. 그 후 첫 번째 고루틴은 b를 얻으려 하고, 두 번째 고루틴은 a를 얻으려 한다. 서로가 서로의 자물쇠를 쥐고 절대로 놓아주지 않으니, 톱니바퀴가 완전히 맞물려 멈춰버린 셈이다.

데드락의 4가지 성립 조건

1971년 에드거 코프먼은 데드락이 발생하려면 다음 4가지 조건이 동시에 충족되어야 함을 밝혀냈다. 반대로 말하면, 이 중 하나라도 깨뜨릴 수 있다면 데드락을 완벽히 예방할 수 있다.

- 상호 배제: 자원은 한 번에 한 프로세스만 독점할 수 있어야 한다.
- 조건 대기: 자원을 이미 쥔 채로, 또 다른 자원을 얻기 위해 대기해야 한다.
- 선점 불가: 남이 쥐고 있는 자원을 강제로 뺏아올 수 없어야 한다.
- 순환 대기: 프로세스들이 서로가 서로를 꼬리를 물고 기다리는 원 형태를 이루어야 한다.

(2) 라이브락(Livelock)

라이브락은 프로그램이 멈추지 않고 무언가 활발하게 연산을 수행하고는 있지만, 실질적으로는 프로그램의 상태를 앞으로 단 1밀리미터도 전진시키지 못하는 상태를 말한다.

좁은 복도에서 반대편 사람과 마주쳤을 때를 상상해보자. 내가 비켜주려고 왼쪽으로 움직였는데 상대방도 똑같이 왼쪽으로 비킨다. 미안해서 서로 오른쪽으로 움직였더니 또 똑같이 움직인다. 이 춤판이 영원히 반복된다면! 이것이 바로 라이브락이다.

실제 라이브락이 발생하는 핵심 메커니즘을 코드로 확인하자. 복도 걷기 알고리즘이다.

// (중략 - 타이밍과 속도를 맞추주는 도구들)

```
tryDir := func(dirName string, dir *int32, out *bytes.Buffer) bool {
    fmt.Fprintf(out, "%v", dirName)
    atomic.AddInt32(dir, 1) // 1. "나 이 방향으로 갈게요"라고 선언
    takeStep()             // 2. 발걸음을 맞추기 위해 잠시 대기

    if atomic.LoadInt32(dir) == 1 { // 다른 사람이 이 방향에 없다면 통과!
        fmt.Fprint(out, ". Success!")
        return true
    }

    takeStep()
    atomic.AddInt32(dir, -1) // 3. 앗, 상대방도 이 방향으로 왔네? 양보하기 위해 뒤로 한 걸
    음 후퇴
    return false
}

walk := func(walking *sync.WaitGroup, name string) {
    var out bytes.Buffer
    defer func() { fmt.Println(out.String()) }()
    defer walking.Done()

    fmt.Fprintf(&out, "%v is trying to scoot:", name)
    for i := 0; i < 5; i++ { // 무한 루프를 방지하기 위해 딱 5번만 시도하도록 제한함
        if tryLeft(&out) || tryRight(&out) { return }
    }
    fmt.Fprintf(&out, "\n%v tosses her hands up in exasperation!", name)
}
```

- 원인 분석:

앨리스와 바바라는 서로 데드락(멈춤)을 피하기 위해 아주 배려심 넘치게 코딩되었다.
“내가 가려는 길에 상대가 있으면 양보하자”하고 말이다. 하지만 조율되지 않은 상태에서 동시에 비켜주는 행동을 반복하다 보니, 서로 길을 막고 비켜 주기를 무한히 반복하다 지쳐 쓰러지게 되었다.

- 라이브락이 무서운 이유:

데드락은 CPU 점유율이 0%가 되며 완전히 멈추기 때문에 바로 타가 난다.
하지만 라이브락은 CPU를 100% 터트리며 엄청나게 바쁘게 일하는 것처럼 보이기 때문에, 시스템 모니터링만으로는 버그를 찾아내기가 매우 어렵다.

(3) 기아 상태(Starvation)

기아 상태는 어떤 동시성 프로세스가 자신이 일하는 데 필요한 자원(공유 자물쇠, CPU, 메모리 등)을 계속해서 배정받지 못해 굶어 죽는 상태를 의미한다.

앞서 본 라이브락은 모든 프로세스가 평등하게(?) 자원을 못 나눠 가져서 다 같이 굶어 죽는 특수한 기아 상태라면, 일반적인 기아 상태는 탐욕스러운 프로세스가 자원을 독점하는 바람에 선량하고 예의 바른 프로세스가 일 효율을 내지 못하는 상황을 뜻한다.

기아 상태를 유발하는 탐욕적 고루틴 예시

```
var wg sync.WaitGroup
var sharedLock sync.Mutex
const runtime = 1*time.Second

greedyWorker := func() {
    defer wg.Done()
    var count int
    for begin := time.Now(); time.Since(begin) <= runtime; {
        sharedLock.Lock()      // 1. 자물쇠를 잠그고
        time.Sleep(3*time.Nanosecond) // 연산을 한 뒤
        sharedLock.Unlock()    // 2. 열자마자 다음 루프에서 자기가 또 바로 잠그려 함!
        count++
    }
    fmt.Printf("Greedy worker executed %v work loops\n", count)
}

politeWorker := func() {
    defer wg.Done()
    var count int
    for begin := time.Now(); time.Since(begin) <= runtime; {
        // 예의 바른 일꾼은 자기가 꼭 필요한 순간에만 쪼개서 자물쇠를 얻으려고 함
        sharedLock.Lock(); time.Sleep(1*time.Nanosecond); sharedLock.Unlock()
        sharedLock.Lock(); time.Sleep(1*time.Nanosecond); sharedLock.Unlock()
        sharedLock.Lock(); time.Sleep(1*time.Nanosecond); sharedLock.Unlock()
        count++
    }
    fmt.Printf("Polite worker executed %v work loops\n", count)
}
```

실행 결과:

Polite worker was able to execute 289777 work loops.
Greedy worker was able to execute 471287 work loops

- 원인 분석:

똑같이 3나노초 동안 일하도록 가상 시뮬레이션을 돌렸음에도 불구하고, 탐욕적인 일꾼이 예의 바른 일꾼보다 거의 2배 가까운 많은일을 처리했다. 탐욕적인 일꾼이 자물쇠(Lock)를 쥐고 놓아주는 범위를 너무 크게 확장해 버린 탓에, 예의 바른 일꾼 고루틴은 계속 자물쇠 차례를 받지 못하고 굼주려야 했던 것이다.

- 해결법과 밸런스

기아 상태를 찾아내려면 시스템의 작업 처리량은 같은 '메트릭(지표) 로그'를 꼼꼼히 기록해야 한다.

(4) 균형 찾기

메모리 접근을 동기화(자물쇠 채우기)할 때는 항상 성능과 공정함 사이에서 예술적인 균형을 잡아야 한다.

자물쇠 범위 스타일	장점	단점
굵은 동기화 (Coarse-grained) 자물쇠를 크게 한 번만 잠그기	락을 자주 켜고 끄지 않아서 자체 성능이 올라감	다른 고루틴을 굼겨 죽이는 기아 상태 유발 확률 높음
세밀한 동기화 (Fine-grained) 임계 영역에만 아주 짧게 잠그기	모든 고루틴에게 기회가 골고루 돌아가서 공정함	자물쇠를 너무 자주 열고 닫느라 비용(오버헤드) 증가

- 강력 추천하는 튜닝 순서

처음 코드를 작성할 때는 무조건 임계 영역에만 타이트하게 자물쇠를 채우는 '세밀한 동기화'로 시작하라. 그러다가 나중에 정말 자물쇠 통제 자체가 성능 병목을 일으킬 때만 범위를 조금씩 넓혀 나가는 것이 소프트웨어 유지보수 측면에서 훨씬 안전하고 올바른 방향이다.

마지막으로, 기아 상태 프로그램 내부의 락뿐만 아니라, CPU 자원, 파일 핸들, DB 커넥션 풀 등을 공유해야 하는 모든 자원에서 일어날 수 있다는 점을 기억하자.

[01-5] 동시성 안전성 판단

(1) 동시성 설계라는 예술

모든 코드의 뒤에는 최소한 한 명의 ‘사람’이 있다. 우리가 지금까지 동시성의 수많은 기술적 난제들을 배웠지만, 새로운 기능을 추가하거나 버그를 고치면서 이 모든 문제를 머릿속으로 통제하고 올바른 결정을 내리는 것은 사람에게 정말 어려운 일이다.

아무것도 없는 빈 도화지에서 동시성 시스템을 설계해야 한다면 다음과 같은 고민에 빠지게 된다.

- 동시성 코드를 호출하는 사람(사용자)에게 어떤 수준까지 내부 구조를 노출해야 할까?
- 사용하기 쉽고 나중에 고치기도 편한 코드를 만드려면 어떤 추상화 테크닉을 써야 할까?
- 이 문제를 푸는 데 가장 적절한 동시성의 강도(수준)는 어느 정도일까?

이러한 고민들을 체계적으로 정리하는 논리적 방법들은 존재하지만, 여전히 완벽한 정답이 없는 ‘예술’의 영역에 가깝다.

(2) 모호한 코드와 소통의 부재

기존에 작성된 코드를 유지보수해야 하는 개발자 입장에서는, 눈앞의 함수가 내부적으로 동시성을 쓰는지 안쓰는지, 그리고 어떻게 써야 안전한지 한눈에 알아채기 힘들다. 다음 함수의 형태(시그니처)를 한 번 보자.

```
// CalculatePi는 begin과 end 사이의 원주율(Pi) 자릿수를 계산합니다.  
func CalculatePi(begin, end int64, pi *Pi)
```

엄청나게 긴 자릿수의 원주율을 계산하는 작업은 동시성(고루틴)으로 돌리기 딱 좋은 작업이다. 하지만 이 함수 딸랑 하나만 보면 개발자는 수많은 의문에 휩싸이게 된다.

- “내가 이 함수를 쓸 때 어떻게 동시성을 적용해야 하지?”
- “내가 직접 고루틴을 여러 개 켜서 이 함수를 여러 번 실행(호출)해야 하나?”
- “인자로 넘겨주는 pi라는 포인터 주소를 보니까 모든 일꾼이 이 하나의 메모리를 직접 건드리는 것 같은데 이 메모리에 자물쇠(Lock)를 채우는 동기화 작업은 내가 해야 할까, 아니면 Pi구조체 내부에서 알아서 해줄까?”

고작 함수 한 줄이 이토록 많은 질문을 던지게 만든다. 적당한 규모의 프로그램만 되어도 이런 모호함이 쌓여 동시성 시스템이 얼마나 복잡하고 무서워질지 상상이 갈거다.

(3) 주석의 힘과 3가지 핵심 요소

이러한 소통의 먹통 상태를 해결하는 첫 번째 강력한 무기는 바로 ‘주석’이다. 만약 위의 코드가 다음과 같이 주석과 함께 쓰여 있다면 어떨까?

```
// CalculatePi는 begin과 end 사이의 원주율(Pi) 자릿수를 계산합니다.  
//  
// [내부 동작 설명]  
// CalculatePi는 내부적으로 FLOOR((end-begin)/2)만큼의 동시성 프로세스를  
// 생성하여 재귀적으로 CalculatePi를 호출합니다.  
// pi 변수에 값을 저장할 때의 동기화(자물쇠 처리)는 Pi 구조체 내부에서  
// 안전하게 처리되므로 안심하셔도 됩니다.  
func CalculatePi(begin, end int64, pi *Pi)
```

이 주석 한 줄 덕분에 우리는 이제 마음 편하게 그냥 호출하면 된다는 것을 알게 되었다. 동시성이나 메모리 오염 걱정을 할 필요가 없어진 것이다.

동시성 코드를 작성할 때 주석에 꼭 담아야 할 3가지 요소

1. 동시성의 책임이 누구에게 있는가?(함수를 부르는 사람 vs 함수 내부)
2. 문제를 해결하기 위해 동시성 도구들을 어떻게 배치하고 매핑했는가?
3. 메모리 동기화(자물쇠 처리)의 책임은 누구에게 있는가?

나중에 이 코드를 볼 미래의 나와 동료들을 위해, 동시성이 얽힌 영역에서는 약간 과하다 싶을 정도로 친절하게 이 3가지 관점을 주석으로 적어주는 배려가 필요하다.

(4) 코드 설계(시그니처)를 통한 신호 전달

애초에 함수 모양 자체가 모호하다는 것은 우리가 설계를 잘못했다는 신호일 수도 있다. 인자로 포인터 변수(*Pi)를 넘겨서 내부 상태를 바꾸는 대신, 부작용이 없는 순수 함수 형태로 바꾸면 어떨까?

```
func CalculatePi(begin, end int64) []uint
```

이 함수는 계산된 값들의 슬라이스([]uint)를 그냥 결과로 반환한다. 공유하는 메모리가 아예 없기 때문에 최소한 “내가 자물쇠(Lock)를 채워야 하나?”라는 동기화 질문은 원천 차단된다. 하지만 여전히 내부에서 동시성을 쓰고 있는지는 알 수 없다.

Go언어의 핵심 무기인 채널(Channel)을 활용해 한 단계 더 명확하게 시그니처를 고쳐보겠다.

```
func CalculatePi(begin, end int64) <-chan uint
```

여기서 <-chan uint라는 채널 반환 타입이 등장한다.

이 모양을 보는 순간 Go 개발자들은 강력한 힌트를 얻는다. “아하! 이 함수는 내부적으로 최소한 하나 이상의 고루틴을 띄워서 백그라운드로 계산을 하고 있겠구나! 그리고 결과가 나오는데로 이 채널에 쏙쏙 던져주겠구나. 내가 굳이 고루틴을 새로 만들어서 감쌀 필요가 없겠네!”하고 말이다.

물론 이렇게 설계를 바꿀 때마다 메모리 할당이나 채널 통신 비용 같은 ‘성능 파급적 효과’가 따르게 되므로, 우리는 다시 한번 ‘명확성(가독성)’과 ‘성능’ 사이의 저울질이라는 본질적인 문제로 돌아오게 된다.

혼자 짜는 코드도 이런데, 팀 단위 프로젝트로 규모를 키우면 이 소통의 왜곡과 복잡성은 정말 어마어마한 골칫거리가 된다.

(5) 결론

여기까지 들으면 동시성이 정말 절망적이고 무서운 숙제처럼 느껴질 것이다. 하지만 정말 다행인 소식은, Go언어가 바로 이러한 인간적인 소통의 한계와 설계의 어려움을 해결하기 위해 태어난 언어라는 점이다. Go 언어는 언어 자체적으로 극단적인 단순함과 가독성을 지향한다. Go가 제안하는 방식으로 동시성 코드를 짜다 보면, 나도 모르게 올바르고, 결합하기 좋으며, 수평 확장이 시운 안전한 코드를 완성하게 된다.

오히려 Go의 동시성 모델을 이해하고 나면, 우리가 해결하려는 현실 세계의 복잡한 문제들을 훨씬 더 많고 명쾌하게 코드로 표현할 수 있게 된다.

[01-6] 복잡함 속에서 단순함

(1) 절망 속에서 찾는 희망

지금까지 제가 동시성 프로그래밍의 어두운 단면(데이터 경쟁, 데드락, 기아 상태 등)을 너무 잔인하게 내려가서 겁을 먹을지도 모른다. 동시성이 컴퓨터 과학에서 가장 까다로운 영역 중 하나인 것은 분명하다.

하지만 저는 여러분에게 이 문제들이 결코 난공불락의 요새가 아니라는 희망을 드리고 싶다. Go언어가 제공하는 동시성 도구(기본형)들을 사용하면, 복잡한 동시성 알고리즘을 그 어떤 언어보다 안전하고 명확하게 코드로 표현할 수 있다.

우리가 앞에서 다루었던 수많은 런타임 오류나 소통의 복잡함을 Go 언어가 마법처럼 완벽히 ‘해결’ 해준 것은 아니다. 하지만 개발자가 체감하는 난이도를 혁신적일 정도로 엄청나게 낮추어 준 것은 사실이다. Go 언어가 어떻게 이 놀라운 진보를 이뤄냈는지는 다음장에서 철학적으로 다루고, 여기서는 Go가 구체적으로 우리의 삶을 어떻게 편하게 만들어주는지 가볍게 살펴보겠다.

(2) 메모리 관리의 해방: 저지연 가비지 컬렉터

Go의 핵심 무대 뒤편(Runtime)에서는 우리를 위해 가장 고되고 복잡한 일들을 대신 처리해 준다. 그중 첫 번째가 바로 가비지 컬렉터(Garbage Collector, GC)이다.

개발자들 사이에서는 “언어에 가비지 컬렉터(자동 메모리 청소부)가 있는게 좋은가, 없는게 좋은가?”를 두고 오랜 기간 논쟁이 이어져왔다. GC를 반대하는 사람들은 이렇게 말한다.

“프로그램이 쓰레기를 청소하기 위해 가동을 잠시 멈추는(Stop-the-world) 순간이 존재하는 데, 1분 1초가 급한 실시간 서비스나 예측 가능한 성능이 중요한 프로그램에서 이런 멈춤 현상은 절대 용납할 수 없다!”

물론 일리 있는 지적이다. 하지만 Go 언어 개발진이 이 가비지 컬렉터를 뼈를 깎는 노력으로 개선해 온 덕분에, 이제 대다수의 일반 개발자들은 “GC가 어떻게 돌아가는지” 그 미세한 원리까지 골머리를 썩일 필요가 완전히 사라졌다. Go언어의 가비지 컬렉터가 발전하면서, 청소를 위해 프로그램을 멈추는 시간이 무려 10~100마이크로초(100만 분의 1초 단위) 수준으로 줄어들었기 때문이다.

이게 왜 동시성 프로그래밍에 도움이 될까?

컴퓨터 과학에서 ‘메모리 관리’는 그 자체로도 아주 골치 아픈 영역이다. 그런데 여기에 ‘동시성(여러 고루틴이 얹히는 환경)’까지 결합되면, 메모리를 안전하게 해제하고 관리하는 코드를 버그없이 짜기란 지옥 난이도에 가깝다.

하지만 여러분이 10마이크로초라는 극 미세한 멈춤조차 허용하지 않아야 하는 특수한 0.1%의 개발자가 아니라면, Go언어가 알아서 메모리를 청소해 주므로 여러 고루틴 사이에서 메모리가 꼬일 걱정을 하지 않고 마음 편히 동시성 코드를 작성할 수 있게 된다.

(3) 스레드 관리의 자동화: 고루틴의 멀티플렉싱

Go의 런타임은 우리가 만든 동시성 작업들을 운영체제의 실제 스레드(OS Thread) 위로 알아서 척척 나누어 얹어 주는 일(Multiplexing)을 자동으로 처리한다.

이게 왜 대단한 건지 다른 언어로 웹서버를 만드는 상황과 비교해볼까.

과거의 방식(다른 언어들의 멀티스레드 모델)

만약 다른 언어에서 수많은 사용자의 접속(Connection)을 동시에 처리하는 웹 서버를 만든다고 해보자. 서버가 문을 열기 전에 개발자는 다음과 같은 복잡한 설계를 해야 한다.

1. 실제 운영체제 스레드를 미리 한 가득 만들어 둔다. (이를 스레드 풀, Thread Pool)
2. 새로운 사용자가 들어올 때마다 이 스레드 풀의 비어있는 스레드에 사용자를 조심스럽게 배정한다.
3. 하나의 스레드에 여러 사용자가 몰리면, 모든 사용자가 CPU를 골고루 나눠 쓸 수 있도록 스레드 내부에서 또 루프를 돌리며 스케줄링을 해야 한다.
4. 어떤 사용자의 작업이 너무 길어지면 다른 사용자가 멈추므로, 작업 중간에 멈췄다 갈 수 있는 복잡한 일시정지 논리까지 코드로 짜 넣어야 한다.

Go언어의 방식

Go언어에서는 이 복잡한 과정을 단 한 줄로 끝낸다.

```
// 동시에 실행하고 싶은 로직을 함수로 만들고  
func HandleConnection(conn Connection) { ... }
```

```
// 호출할 때 앞에 go 키워드만 붙여주면 끝!  
go HandleConnection(conn)
```

앞서 수많은 스레드를 만들고, 조율하고, 쪼개고, 분배하던 그 무시무시한 일들을 Go 런타임이 백그라운드에서 완벽하게 자동으로 처리해준다.

이 두 가지 모델 중에서 여러분이 설계를 고민할 때 어떤 모델이 동시성을 더 도입하기 쉽고 만만해 보이나요? 그리고 어떤 방식이 버그 없이 정확하게 작동할 확률이 높을까?
정답은 명확하다.

(4) 거대한 문제를 쉽게 조립하는 채널

Go 언어의 동시성이 가진 또다른 강력한 매력은 ‘결합성’이다.

나중에 채널 섹션에서 자세히 배우겠지만, Go 언어가 제공하는 채널이라는 도구를 사용하면 서로 독립적으로 돌아가는 동시성 프로세스(고루틴)들이 안전하게 데이터를 주고받으며 소통할 수 있는 완벽한 통로를 구축할 수 있다. 자물쇠(Mutex)를 덕지덕지 채우지 않고도, 마치 레고 블록을 조립하듯이 작은 동시성 작업들을 연결해 거대하고 안전한 시스템을 완성할 수 있게 된다.

(5) 결론

지금은 이 모든 메커니즘이 어떻게 돌아가는지 세부적인 원리까지는 몰라도 괜찮다. 이 장에서 전하고 싶었던 진심은, Go 언어가 여러분을 동시성 프로그래밍의 세계로 아주 다정하게 초대하고 있다는 사실이다.

Go는 복잡하고 정교한 지식이 없어도, 누구나 명확하고 성능 좋은 동시성 프로그램을 짤 수 있도록 판을 깔아두었다.

다음 2장에서는 Go언어가 왜 이런 훌륭한 설계를 선택했는지 그 내면에 담긴 ‘동시성의 철학’을 살펴볼 것이다.

만약 철학적인 이야기보다 빨리 실제 작동하는 코드를 맛보고 싶으시다면, 3장으로 점프해도 좋다.